Лабораторная работа 8 Непрерывная интеграция и quality gates

Версия: 2026–2027 учебный год Формат: команда 2–5 человек с индивидуальной сдачей Предшествующая работа: ЛР7 «Сервис предсказаний и контракт API»

1 Цель

Преобразовать локальные проверки проекта в воспроизводимый GitHub Actions pipeline, который автоматически блокирует принятие дефектного изменения, сохраняет диагностические артефакты и даёт проверяемые доказательства личной работы каждого участника.

2 Результаты обучения

Каждый студент должен уметь:

- объяснять события, jobs, steps, actions и runners;
- различать локальную проверку и обязательный remote check;
- настраивать минимальные permissions и отмену устаревших запусков;
- проектировать порядок быстрых и дорогих gates;
- читать log, exit code и JUnit/JSON-отчёт;
- сохранять диагностические artifacts без секретов;
- воспроизводить контролируемый отказ и исправлять корневую причину;
- связывать run, commit, pull request и правило ветки;
- подтверждать личный вклад независимо от результата команды.

3 Командный результат

Команда предоставляет tag `lr8-v1` и:

1. workflow `.github/workflows/ci.yml`;
2. события `pull_request`, push в `main` и ручной `workflow_dispatch`;
3. read-only permission для contents;
4. concurrency с отменой устаревшего run той же ветки;
5. job `quality` с lint, format check, data validation, tests, OpenAPI export и smoke training;
6. job `container`, зависящий от успешного `quality`;
7. image tag по полному `${{ github.sha }}`, без `latest`;
8. diagnostic artifact с JUnit, data-validation и OpenAPI reports;
9. required checks `quality` и `container` для `main`;
10. один намеренно неуспешный и один успешный run на каждого участника;
11. issue, небольшие PR, review и индивидуальные evidence cards.

4 Модель GitHub Actions

Понятие	Назначение	Пример в ЛР8
event	причина запуска	`pull_request`
workflow	файл автоматизации	`ci.yml`
job	изолированный набор шагов	`quality`
step	команда или action	`python -m pytest`
action	переиспользуемая операция	`actions/checkout`
runner	машина выполнения	`ubuntu-latest`
artifact	сохранённый результат run	JUnit и OpenAPI
check	статус job в pull request	`quality`

Каждый job получает чистый runner. Наличие файла на ноутбуке или в другом job нельзя предполагать без явной передачи artifact или повторного построения.

5 События запуска

Workflow запускается:

- при открытии и обновлении pull request;
- при push в `main`, чтобы проверить принятую историю;
- вручную через `workflow_dispatch` для демонстрации и диагностики.

Не используйте `pull_request_target` для исполнения кода из недоверенного fork: этот event имеет иной security context и при ошибочной конфигурации может открыть secrets или write token. Для учебного CI достаточно `pull_request`.

6 Permissions и supply-chain граница

В начале workflow задаётся:

```
permissions:
  contents: read
```

Token получает только необходимое право чтения. Checkout выполняется с `persist-credentials: false`, поскольку последующие steps не должны выполнять push.

Используйте проверенные официальные actions и зафиксированные major versions. На начало подготовки материалов используются `actions/checkout@v7`, `actions/setup-python@v7` и `actions/upload-artifact@v4`. Перед запуском потока преподаватель повторно сверяет поддерживаемые версии и changelog; номер в методичке не является бессрочной гарантией.

В зрелом pipeline action дополнительно закрепляют полным commit SHA. В ЛР8 студент должен объяснить разницу между major tag и immutable SHA.

7 Concurrency

```
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true
```

Если участник быстро отправляет несколько commits в одну ветку, предыдущий run уже не доказывает качество актуального commit. Отмена экономит runner time и уменьшает очередь. Run другой ветки не должен отменяться, поэтому group включает ref.

8 Порядок quality gates

Job `quality` выполняет:

1. checkout;
2. установку Python с pip cache;

3 установку проекта и dev-зависимостей;

4 `ruff check`;

5 `ruff format --check`;

6 проверку data contract;

7 pytest с JUnit report;

8 экспорт OpenAPI;

9 smoke training;

10 загрузку reports как artifact.

Быстрые статические проверки расположены раньше дорогих. Ошибка прекращает дальнейшие обычные steps, но upload выполняется с `if: always()`, чтобы сохранить уже созданную диагностику.

9 Container gate

Job `container` содержит `needs: quality`. Дорогая сборка начинается только после успешных проверок кода, данных и API.

```
- name: Build immutable image tag
  run: docker build --tag mlops-flight-delay:${{ github.sha }} .
```

SHA связывает image с конкретным commit. Tag `latest` изменяем и не является доказательством происхождения. В ЛР8 push в registry ещё не выполняется: публикация без политики доступа и release process преждевременна.

10 Diagnostic artifacts

Workflow сохраняет каталог `reports/` на 14 дней. Минимум:

- `reports/validation/ci.json`;
- `reports/tests/ci.xml`;
- `reports/openapi/ci-openapi.json`.

Artifact предназначен для диагностики и доказательства, а не для постоянного хранения модели. Не загружайте `.env`, tokens, полный production dataset, персональные данные и необработанные request payloads.

Имя включает `github.run_id`, поэтому evidence card однозначно ссылается на run. Студент скачивает artifact и проверяет содержимое, а не ограничивается зелёной иконкой.

11 Проверка CI-контракта

Тест `tests/test_ci_contract.py` защищает ключевые решения workflow: permissions, concurrency, обязательные команды, artifact upload, зависимость jobs и immutable image tag. Он также запрещает `pull_request_target` и `latest`.

Текстовый contract test не доказывает полную корректность YAML и GitHub semantics. Он служит ранней защитой от учебного регресса и дополняется реальным Actions run.

12 Настройка защиты `main`

В branch protection или ruleset настройте:

- изменения только через pull request;
- минимум одно одобрение не от автора;
- разрешение обсуждений до merge;
- required status checks `quality` и `container`;
- запрет force push и удаления `main`;

- отсутствие обхода правил в учебном процессе.

Имена checks должны быть уникальны. Если одинаковое имя появляется в нескольких workflows, GitHub может показывать неоднозначный статус required check.

Если тариф или права не позволяют применить настройку, команда документирует ограничение и демонстрирует эквивалентный обязательный процесс. Это снижает полноту автоматизации и должно быть явно отражено в evidence.

13 Порядок выполнения

Этап 1. Подготовка

1. Синхронизируйте `main` и создайте issue.
2. Создайте отдельную ветку.
3. Выполните локально все команды из PR template.
4. Убедитесь, что generated reports не содержат секретов.

Этап 2. Положительный pipeline

1. Добавьте workflow.
2. Создайте pull request.
3. Дождитесь `quality` и `container`.
4. Откройте каждый step, зафиксируйте run ID и commit SHA.
5. Скачайте artifact и проверьте три обязательных файла.

Этап 3. Индивидуальный controlled failure

Каждый участник в собственной ветке вводит один небольшой обратимый дефект, заранее указывает ожидаемый упавший step и отправляет commit. После красного run участник фиксирует ссылку и диагностику, исправляет причину отдельным commit и получает зелёный run.

Этап 4. Branch protection

Настройте required checks и покажите, что merge недоступен при красном `quality` или `container`. После успешных checks и review выполните merge по принятой стратегии.

Этап 5. Фиксация версии

На принятом commit создайте tag `lr8-v1`. Ссылки evidence должны вести на runs соответствующей ветки и итоговый commit.

14 Варианты контролируемого отказа

Дефект	Ожидаемый step	Диагностическое доказательство
нарушение Ruff	Lint	файл, строка и rule code
неотформатированный файл	Check formatting	diff formatter
неверный тип в данных	Validate data contract	JSON report и violation
сломанный инвариант теста	Run tests	test node ID и JUnit failure
удалённый response schema	Run tests или Export OpenAPI	упавший contract test
синтаксическая ошибка Dockerfile	Build immutable image tag	build stage и error
отсутствующий файл в build context	container	COPY error

Участники одной команды выбирают разные дефекты. Нельзя подменять эксперимент ручной отменой run или временным отключением required check.

15 Диагностика красного run

Используйте последовательность:

1. проверить event, branch, commit SHA и отмену устаревшего run;
2. найти первый упавший step, а не последний пропущенный;
3. прочитать команду и exit code;
4. открыть аннотацию или report;
5. воспроизвести команду локально;
6. сформулировать корневую причину одним проверяемым предложением;
7. внести минимальное исправление;
8. сопоставить новый run с исправляющим commit.

Кнопка Re-run полезна для внешней нестабильности, но не является исправлением детерминированного дефекта.

16 Матрица командных ролей

Для команды 2–5 человек роли объединяются или распределяются:

- владелец quality job;
- владелец container job;
- владелец diagnostic artifacts;
- владелец branch rules и PR process;
- reviewer CI/security contract.

В следующей ЛР роли ротируются. Один участник не должен единолично написать workflow и раздать остальным одинаковые screenshots.

17 Индивидуальная сдача участника команды

Командный результат: tag `lr8-v1`, защищённый `main`, успешные checks и diagnostic artifact.

Поле	Что указать
Студент	ФИО, группа, команда, GitHub-логин
Лабораторная	ЛР8, итоговый commit/tag
Роль	роль в ЛР8 и отличие от ЛР7
Задача	issue с наблюдаемым критерием приёмки
Личный артефакт	workflow step, test, report, правило или review
Негативный run	ссылка, run ID, commit и ожидаемый failed step
Диагностика	первая полезная ошибка, exit code/report и причина
Исправление	отдельный commit и успешный run
Artifact	имя, срок хранения и проверенные файлы
Инженерное решение	порядок gates или permission и альтернатива
Риск	CI/supply-chain риск и мера
Использование ИИ	инструмент и самостоятельно проверенные утверждения либо отсутствие использования

Минимальный допуск: личная issue закрыта через PR, negative run сохранён, причина объяснена, исправление подтверждено green run, review выполнен другим участником, итоговые required checks относятся к принимаемому commit.

18 Защита

За 3–4 минуты участник:

1. показывает свою issue и изменение;
2. открывает намеренно неуспешный run;
3. называет первый упавший step и корневую причину;
4. открывает исправляющий commit и зелёный run;
5. скачивает или открывает свой report из artifact;
6. объясняет, почему `container` не стартовал до `quality`;
7. показывает required check и невозможность merge при отказе;
8. объясняет один риск token/action/artifact.

Преподаватель может попросить определить, какой run относится к HEAD pull request, изменить data contract, удалить response schema или объяснить, почему re-run без commit не исправляет код.

19 Повторная сдача и стоп-факторы

Повторная сдача выполняется через новую issue или переоткрытый PR, отдельный исправляющий commit, новые runs и обновлённую evidence card. История первой попытки сохраняется.

Стоп-факторы:

- workflow запускается только вручную;
- merge выполнен с красным required check;
- `contents: write` задан без обоснования;
- используется `pull_request_target` для выполнения student code;
- image помечен только `latest`;
- reports или logs содержат secret/персональные данные;
- negative run общий на команду либо создан отменой;
- студент не может связать run с commit;
- успешный статус получен отключением проверки;
- evidence отредактировано вручную без происхождения.

20 Критерии оценивания

Критерий	Баллы	Полное выполнение
Workflow и события	2	PR, main, manual; permissions и concurrency корректны
Quality gates	2	lint, format, data, tests, OpenAPI, training выполняются
Container и происхождение	1	зависимость от quality и tag по SHA
Artifacts и диагностика	1	отчёты сохраняются и читаются участником
Защита ветки и review	1	required checks и содержательное review
Индивидуальный failure/fix	3	отдельные runs, причина, исправление и устная защита

Максимум: 10 баллов. Рекомендуемый порог зачёта: 6 баллов при отсутствии стоп-факторов.

21 Чек-лист

- □ Команда состоит из 2–5 человек, роли ротированы.

- □ Workflow запускается на pull request, push в main и вручную.
- □ Token имеет только `contents: read`.
- □ Устаревший run ветки отменяется concurrency policy.
- □ `quality` содержит все обязательные проверки.
- □ JUnit, validation и OpenAPI входят в artifact.
- □ `container` зависит от `quality`.
- □ Image tag содержит полный commit SHA.
- □ `quality` и `container` являются required checks.
- □ Каждый участник создал собственный negative run.
- □ Каждый negative run исправлен отдельным commit.
- □ Evidence card содержит ссылки, run ID и диагностику.
- □ Итоговая версия отмечена tag `lr8-v1`.

22 Связь с ЛР9

В ЛР9 pipeline будет расширен публикацией версионированного image и безопасной работой с registry. ЛР8 создаёт доверенную границу: публикация возможна только после проверок кода, данных, API и container build.

23 Официальные источники

- GitHub Actions workflow syntax: https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax
- GitHub Actions concurrency: https://docs.github.com/en/actions/concepts/workflows-and-actions/concurrency
- Workflow artifacts: https://docs.github.com/en/actions/concepts/workflows-and-actions/workflow-artifacts
- Protected branches: https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches
- actions/checkout releases: https://github.com/actions/checkout/releases
- actions/setup-python releases: https://github.com/actions/setup-python/releases
- actions/upload-artifact: https://github.com/actions/upload-artifact